

```

--
=====
-- BANCO DE DADOS REMAMA / ONCOFIT
-- DDL DOCUMENTADO - FASE 1 (SCHEMA + DADOS DE REFERÊNCIA +
INTEGRIDADE)
--
=====
--
-- SOBRE ESTE ARQUIVO
-- Este é o schema completo do banco de dados do projeto REMAMA/OncoFit
-- (programa de reabilitação por dragon boat + treinamento funcional
-- para sobreviventes de câncer de mama).
--
-- O que este arquivo NÃO contém, de propósito:
-- - ENABLE ROW LEVEL SECURITY / CREATE POLICY
--   (fica reservado para a Fase 2, depois de confirmados os papéis
--   de acesso - admin / pesquisador / voluntária - com a
--   administração do Remama)
--
-- Convenção de nomenclatura usada no banco todo:
-- - Tabelas de domínio/lookup (valores fixos, poucas linhas,
--   alteram raramente): PK manual (INT PRIMARY KEY) quando o valor
--   numérico tem significado de negócio (ex: id_status = 1 sempre
--   é "Ativa"); PK por IDENTITY quando a ordem não importa.
-- - Tabelas transacionais (crescem continuamente): sempre PK por
--   IDENTITY.
-- - Todo CHECK reflete uma regra de negócio ou uma escala validada
--   (ex: PSE segue a Escala de Borg CR-10, de 0 a 10).
--
=====

--
=====
-- SEÇÃO 1: TABELAS DE DOMÍNIO (LOOKUP)
--
=====
-- Todas as tabelas desta seção existem para evitar texto livre em
-- campos que, na prática, só assumem um conjunto fixo e conhecido de
-- valores. Sem elas, cada usuária do sistema poderia digitar o mesmo
-- conceito de formas diferentes (ex.: "Oncofit", "ONCOFIT", "Onco
-- Fit"), quebrando qualquer agrupamento/relatório feito em cima
-- desse dado.
--
=====

-- status_participante: situação da participante DENTRO DO REMAMA COMO
-- UM TODO (não confundir com o status por programa individual, que

```

```

-- fica em inscricao_programa.data_desligamento - ver SEÇÃO 8, a
-- trigger, para o detalhe de como os dois se relacionam).
--
-- PK manual (não-IDENTITY): o valor numérico é usado como constante
-- de código em outras partes do sistema (ex: "WHERE id_status = 1"
-- na trigger da Seção 8), então precisa ser fixo e previsível.
CREATE TABLE status_participante (
  id_status INT PRIMARY KEY,
  nome_status VARCHAR(20) NOT NULL,
  situacao BOOLEAN NOT NULL DEFAULT TRUE -- TRUE = participante tem algum
vínculo ativo; FALSE = nenhum vínculo ativo
);

```

```

-- estado_civil: PK por IDENTITY porque o valor numérico não é usado
-- como constante em nenhuma regra de negócio (ao contrário de
-- status_participante) - a ordem de inserção não importa aqui.
CREATE TABLE estado_civil (
  id_estado_civil INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  descricao VARCHAR(20) NOT NULL
);

```

```

-- programa: os 3 programas oferecidos pelo Remama.
-- PK manual e fixa: 1 = Remama, 2 = OncoFit, 3 = Ambos (participante
-- inscrita nos dois programas ao mesmo tempo, contabilizada como uma
-- única inscrição em vez de duas linhas em inscricao_programa).
CREATE TABLE programa (
  id_programa INT PRIMARY KEY,
  nome_programa VARCHAR(50) NOT NULL,
  dias_semana VARCHAR(100),
  horario TIME
);

```

```

-- tipo_comorbidade: condições de saúde acompanhadas nas participantes
-- (hipertensão, diabetes etc.). Tabela de domínio clínico -
-- lista deve ser validada com a equipe de saúde do Remama.
CREATE TABLE tipo_comorbidade (
  id_tipo_comorbidade INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome_comorbidade VARCHAR(100) NOT NULL
);

```

```

-- tipo_treinamento: modalidades de treino oferecidas (remada,
-- funcional, alongamento etc.).
CREATE TABLE tipo_treinamento (
  id_tipo_treinamento INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome_treinamento VARCHAR(50) NOT NULL
);

```

```

-- status_presenca: os possíveis resultados de uma chamada de

```

```

-- frequência (Presente / Falta / Justificada).
CREATE TABLE status_presenca (
    id_status_presenca INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    sigla CHAR(1) NOT NULL,
    descricao VARCHAR(20) NOT NULL
);

-- cargo_participante: papel da participante dentro de um programa
-- específico (não confundir com nivel_acesso, da Seção 6, que é
-- sobre permissão de sistema - este aqui é sobre função no barco/
-- na equipe).
-- PK manual e fixa: 1 = Aluna, 2 = Gerente, 3 = Capitã, 4 = Dragonete.
CREATE TABLE cargo_participante (
    id_cargo INT PRIMARY KEY,
    nome_cargo VARCHAR(30) NOT NULL
);

-- funcoes: cargos da equipe profissional/técnica (não das
-- participantes) - referenciada por funcionarios.
CREATE TABLE funcoes (
    id_funcao INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome_funcao VARCHAR(100) NOT NULL
);

-- tipo_cancer: subtipos histológicos de câncer de mama.
-- Criada para substituir o que originalmente era um campo de texto
-- livre (ficha_medica.tipo_cancer VARCHAR), pelo mesmo motivo de
-- padronização das demais tabelas de domínio desta seção.
-- Lista sugerida a partir dos subtipos mais comuns - deve ser
-- validada com a equipe clínica antes de uso em produção.
CREATE TABLE tipo_cancer (
    id_tipo_cancer INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome_cancer VARCHAR(100) NOT NULL,
    cid_o VARCHAR(10) -- opcional: código CID-O, para rastreabilidade clínica formal
);

--
=====
-- SEÇÃO 2: TABELA PRINCIPAL (PARTICIPANTE)
--
=====
-- Entidade central do banco - praticamente toda outra tabela
-- transacional referencia participante via id_participante.
--
=====

CREATE TABLE participante (

```

```

id_participante INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

-- Identificadores únicos do mundo real - UNIQUE previne cadastro
-- duplicado da mesma pessoa.
cpf CHAR(11) UNIQUE NOT NULL,
numero_carteirinha VARCHAR(20) UNIQUE,

-- FK para status_participante: qual é a situação global da
-- participante. Este campo é MANTIDO AUTOMATICAMENTE pela
-- trigger da Seção 8 - não deve ser alterado manualmente via
-- UPDATE direto (ver comentário na trigger para o motivo).
id_status INT REFERENCES status_participante(id_status),

data_inscricao TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
nome_completo VARCHAR(150),
nome_social VARCHAR(150),
data_nascimento DATE NOT NULL,

-- CHECK simples: evita digitação de qualquer caractere fora de F/M.
sexo CHAR(1) DEFAULT 'F' CHECK (sexo IN ('F', 'M')),

id_estado_civil INT REFERENCES estado_civil(id_estado_civil),
possui_filhos BOOLEAN DEFAULT FALSE,
quantidade_filhos SMALLINT DEFAULT 0,
profissao VARCHAR(100),
trabalha_atualmente BOOLEAN,

-- CHECK com lista fechada: faixa de renda familiar, campo usado
-- em análises socioeconômicas do perfil das participantes.
renda_familiar_faixa VARCHAR(20) CHECK (renda_familiar_faixa IN ('Até 1 SM', '1-3
SM', 'Mais de 3 SM')),

foto_url VARCHAR(500),
observacoes TEXT
);

--
=====
-- SEÇÃO 3: TABELAS QUE DEPENDEM DE PARTICIPANTE (1:1 e 1:N)
--
=====
-- Todas as FKs desta seção apontam para participante(id_participante).
-- A ordem de criação importa aqui: participante precisa existir antes.
--
=====

-- endereco: relação 1:N (uma participante pode, em teoria, ter mais

```

-- de um endereço registrado ao longo do tempo).

```
CREATE TABLE endereco (  
  id_endereco INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  id_participante INT NOT NULL REFERENCES participante(id_participante),  
  cep CHAR(8) NOT NULL,  
  logradouro VARCHAR(150) NOT NULL,  
  numero VARCHAR(10) NOT NULL,  
  complemento VARCHAR(100),  
  bairro VARCHAR(50) NOT NULL,  
  cidade VARCHAR(50) NOT NULL,
```

-- CHECK com as 27 UFs do Brasil: evita erro de digitação

-- (ex: "sp" minúsculo, "SPP", etc.) que quebraria qualquer

-- agrupamento geográfico.

```
uf CHAR(2) NOT NULL CHECK (uf IN (  
  'AC','AL','AP','AM','BA','CE','DF','ES','GO','MA','MT','MS','MG',  
  'PA','PB','PR','PE','PI','RJ','RN','RS','RO','RR','SC','SP','SE','TO'  
))
```

```
);
```

-- contato: dados de contato + contato de emergência (obrigatório,

-- por se tratar de programa de atividade física com público de

-- risco à saúde).

```
CREATE TABLE contato (  
  id_contato INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  id_participante INT NOT NULL REFERENCES participante(id_participante),  
  telefone_fixo VARCHAR(14),  
  celular VARCHAR(15) NOT NULL,  
  contato_emergencia_nome VARCHAR(100) NOT NULL,  
  contato_emergencia_tel VARCHAR(15) NOT NULL,  
  contato_emergencia_parentesco VARCHAR(30)  
);
```

-- dados_antropometricos: série histórica de medidas corporais -

-- relação 1:N intencional (uma linha por medição ao longo do tempo,

-- não um cadastro único sobrescrito).

```
CREATE TABLE dados_antropometricos (  
  id_antropometrico INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  id_participante INT NOT NULL REFERENCES participante(id_participante),  
  data_medicao TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP NOT NULL,  
  peso_kg DECIMAL(5,2) NOT NULL,  
  altura_m DECIMAL(3,2) NOT NULL,  
  cintura_cm DECIMAL(5,2),  
  quadril_cm DECIMAL(5,2)  
);
```

-- avaliacao: bateria de testes físicos periódicos (força, marcha,

-- flexibilidade, esforço percebido). Relação 1:N - numero_avaliacao +

```

-- ano identificam qual "rodada" de testes é aquela.
--
-- HISTÓRICO DE CORREÇÃO DESTA TABELA (documentado para rastreabilidade):
-- O desenho original tinha mmss_id_kg/mmss_le_kg como DECIMAL(5,2),
-- nome sugerindo medição em quilos. Ao comparar com a planilha real
-- de testes (REMAMA_Base_Testes_2026.xlsx), constatou-se que o dado
-- efetivamente coletado é NÚMERO DE REPETIÇÕES, não carga em kg.
-- As colunas foram renomeadas para mmss_id_reps/mmss_le_reps e o
-- tipo alterado para INT.
CREATE TABLE avaliacao (
  id_avaliacao INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  id_participante INT NOT NULL REFERENCES participante(id_participante),
  numero_avaliacao INT NOT NULL,
  ano INT NOT NULL,
  data_avaliacao TIMESTAMPTZ NOT NULL,

  -- Testes de força - CHECK de faixa evita valor negativo ou
  -- claramente incompatível com uma contagem de repetições humana.
  sentar_levantar_reps INT CHECK (sentar_levantar_reps BETWEEN 0 AND 200),
  mmss_id_reps INT CHECK (mmss_id_reps BETWEEN 0 AND 200), -- membro superior
  -- direito, em repetições
  mmss_le_reps INT CHECK (mmss_le_reps BETWEEN 0 AND 200), -- membro superior
  -- esquerdo, em repetições
  mmss_id_dominante BOOLEAN,
  mmss_le_dominante BOOLEAN,

  marcha_passadas INT CHECK (marcha_passadas BETWEEN 0 AND 300),

  -- CHECK com lista fechada: classificação qualitativa do teste de
  -- flexibilidade, com valores fixos usados no protocolo do Remama.
  classificacao_flexibilidade VARCHAR(50) CHECK (classificacao_flexibilidade IN
('Excelente', 'Boa', 'Regular', 'Frac')),

  amplitude_braco_cm DECIMAL(5,2),

  -- Campos de observação qualitativa em texto livre. Sem CHECK
  -- porque não há, até o momento, um protocolo padronizado de
  -- preenchimento documentado para eles (ao contrário dos demais
  -- campos desta tabela).
  movimentacao_tronco TEXT,
  equilibrio TEXT,
  cond_aerobico TEXT,
  forca TEXT,

  -- PSE = Percepção Subjetiva de Esforço, medida DURANTE a bateria
  -- de testes físicos (diferente do pse de registro_frequencia, que
  -- mede o esforço em um treino do dia a dia - ver comentário na
  -- Seção 4 para a distinção completa).

```

```
pse INT CHECK (pse BETWEEN 0 AND 10) -- Escala de Borg CR-10 (0 = repouso total,
10 = esforço máximo)
);
```

```
-- ficha_medica: dados clínicos sensíveis (LGPD - dado de saúde é
-- categoria especial). UNIQUE em id_participante força a relação
-- 1:1 - cada participante tem no máximo uma ficha médica.
```

```
CREATE TABLE ficha_medica (
    id_ficha INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_participante INT NOT NULL UNIQUE REFERENCES participante(id_participante),
    atestado_medico_url VARCHAR(500),
    doc_oncologico_url VARCHAR(500),
    data_diagnostico TIMESTAMPTZ,
    pos_menopausa BOOLEAN,
```

```
-- FK para tipo_cancer (ver histórico de correção no comentário
-- da tabela tipo_cancer, na Seção 1) - substitui o antigo campo
-- de texto livre.
```

```
id_tipo_cancer INT REFERENCES tipo_cancer(id_tipo_cancer),
```

```
mamas_afetadas VARCHAR(5) CHECK (mamas_afetadas IN ('LD', 'LE', 'Ambas')),
recidiva BOOLEAN DEFAULT FALSE,
data_recidiva TIMESTAMPTZ,
locais_afetados TEXT,
tratamentos_realizados TEXT
```

```
);
```

```
-- comorbidade: relação N:N materializada entre participante e
-- tipo_comorbidade (uma participante pode ter várias comorbidades;
-- uma comorbidade é compartilhada por várias participantes).
```

```
CREATE TABLE comorbidade (
    id_comorbidade INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_participante INT NOT NULL REFERENCES participante(id_participante),
    id_tipo_comorbidade INT REFERENCES tipo_comorbidade(id_tipo_comorbidade),
    toma_medicamento BOOLEAN,
    qual_medicamento TEXT
```

```
);
```

```
-- treinamento: registra a CONCLUSÃO de uma especialização de
-- treinamento por uma participante (não confundir com uma sessão de
-- treino do dia a dia, que é o que registro_frequencia rastreia).
```

```
CREATE TABLE treinamento (
    id_treinamento INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_participante INT NOT NULL REFERENCES participante(id_participante),
    id_tipo_treinamento INT NOT NULL REFERENCES
tipo_treinamento(id_tipo_treinamento),
    data_conclusao TIMESTAMPTZ
```

```
);
```

-- evento: participação em competições/eventos externos (ex:
-- campeonatos de dragon boat, corridas beneficentes).

```
CREATE TABLE evento (  
    id_evento INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    id_participante INT NOT NULL REFERENCES participante(id_participante),  
    nome_evento VARCHAR(150) NOT NULL,  
    data_evento TIMESTAMPTZ NOT NULL,  
    resultado TEXT  
);
```

-- inscricao_programa: vínculo de uma participante a um programa
-- específico (Remama / OncoFit / Ambos), com cargo e datas.

--
-- ESTA TABELA É METADE DA LÓGICA DA TRIGGER DA SEÇÃO 8. O campo
-- data_desligamento aqui é POR PROGRAMA (granularidade fina) e é
-- diferente de participante.id_status (granularidade global) - ver
-- o comentário completo na trigger para o relacionamento entre os
-- dois.

```
CREATE TABLE inscricao_programa (  
    id_inscricao INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    id_participante INT NOT NULL REFERENCES participante(id_participante),  
    id_programa INT NOT NULL REFERENCES programa(id_programa),  
    id_cargo INT REFERENCES cargo_participante(id_cargo),  
    turma VARCHAR(20),  
    data_inscricao TIMESTAMPTZ DEFAULT CURRENT_DATE,  
    especializacao_leme BOOLEAN DEFAULT FALSE,  
    especializacao_tambor BOOLEAN DEFAULT FALSE,
```

-- NULL = inscrição ainda ativa. Data preenchida = participante
-- saiu DESTES programa especificamente (pode continuar ativa em
-- outro programa, se tiver mais de uma inscrição).
data_desligamento DATE NULL

);

--

=====

-- SEÇÃO 4: TABELA DE FREQUÊNCIA

--

=====

-- registro_frequencia: uma linha por chamada/presença registrada.
-- Pode estar ligada a um treinamento OU a um evento (nunca aos dois,
-- nunca a nenhum) - ver o CHECK ao final da tabela.

```
CREATE TABLE registro_frequencia (  
    id_frequencia INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    id_participante INT NOT NULL REFERENCES participante(id_participante),
```



```
id_status_presenca INT NOT NULL REFERENCES
status_presenca(id_status_presenca),
```

```
-- Ambas nullable de propósito: cada registro pertence a um
-- treinamento OU a um evento, nunca aos dois simultaneamente
-- (regra garantida pelo CHECK no final da tabela).
id_treinamento INT NULL REFERENCES treinamento(id_treinamento),
id_evento INT NULL REFERENCES evento(id_evento),
```

```
data_registro DATE NOT NULL DEFAULT CURRENT_DATE,
justificativa_url VARCHAR(500),
```

```
-- Frequência cardíaca antes/depois do treino - CHECK de faixa
-- fisiologicamente plausível para humanos (evita erro de
-- digitação grosseiro, ex: "750" em vez de "75").
fc_antes INT CHECK (fc_antes BETWEEN 30 AND 220),
fc_depois INT CHECK (fc_depois BETWEEN 30 AND 220),

-- Nota: pse foi REMOVIDO desta tabela e existe apenas em
-- avaliacao.pse. Motivo: o PSE que a Remama registra na prática
-- é o esforço percebido DURANTE A AVALIAÇÃO FÍSICA (bateria de
-- testes), não do treino cotidiano - então o campo pertence
-- semanticamente a avaliacao, não a este registro de presença.
```

```
-- Regra de negócio: todo registro de frequência precisa estar
-- vinculado a ALGUMA atividade concreta (um treino ou um evento).
-- Um registro sem nenhum dos dois não tem contexto - não se sabe
-- do que é a presença/falta.
CHECK (id_treinamento IS NOT NULL OR id_evento IS NOT NULL)
```

```
);
```

```
--
```

```
=====
```

```
-- SEÇÃO 5: TABELAS DE FUNCIONÁRIOS E AUDITORIA
```

```
--
```

```
=====
```

```
-- funcionarios: cadastro profissional da equipe técnica (educador
-- físico, fisioterapeuta, administração etc.).
-- Nota: NÃO tem coluna de senha - a autenticação de funcionários é
-- centralizada em usuario_acesso (Seção 6), para não ter duas fontes
-- de senha para a mesma pessoa.
```

```
CREATE TABLE funcionarios (
  id_funcionario INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nome_funcionario VARCHAR(150) NOT NULL,
  id_funcao INT NOT NULL REFERENCES funcoes(id_funcao),
  email VARCHAR(100) UNIQUE NOT NULL
```

```

);

-- auditoria_sistema: trilha de auditoria de alterações no sistema
-- (quem fez o quê, quando, e o valor antes/depois em JSON).
CREATE TABLE auditoria_sistema (
    id_audit BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    data_evento TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    id_funcionario INT NOT NULL REFERENCES funcionarios(id_funcionario),
    tabela_afetada VARCHAR(50) NOT NULL,
    registro_id INT NOT NULL,

    -- CHECK com lista fechada: evita "update"/"Update"/"UPD" como
    -- três valores diferentes para o mesmo evento, o que quebraria
    -- qualquer relatório de auditoria feito sobre esta coluna.
    acao VARCHAR(10) NOT NULL CHECK (acao IN ('INSERT', 'UPDATE', 'DELETE')),

    dados_antigos JSONB,
    dados_novos JSONB,
    ip_origem VARCHAR(45)
);

--
=====
-- SEÇÃO 6: AUTENTICAÇÃO E CONTROLE DE ACESSO
--
=====
-- Estas duas tabelas são a base para o RLS que será implementado na
-- Fase 2. Aqui só criamos a estrutura de dados; ENABLE ROW LEVEL
-- SECURITY e CREATE POLICY ficam para depois, quando os papéis forem
-- confirmados com a administração do Remama.
--
-- Arquitetura pensada para banco de produção RODANDO FORA DO
-- SUPABASE (Postgres puro), por isso NÃO depende de auth.uid() nem
-- de auth.users (exclusivos do Supabase Auth / GoTrue). Em vez
-- disso, o backend da aplicação autentica o usuário e injeta a
-- identidade na sessão do Postgres via SET LOCAL - as políticas de
-- RLS (fase 2) vão ler essa informação com current_setting().
--
=====

-- nivel_acesso: os três papéis de sistema definidos para o projeto.
CREATE TABLE nivel_acesso (
    id_nivel_acesso INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome_nivel VARCHAR(30) NOT NULL UNIQUE -- 'admin', 'pesquisador', 'voluntario'
);

-- usuario_acesso: elo entre o login (email/senha) e a pessoa real no

```

```

-- banco - seja uma participante (via id_participante) ou um membro
-- da equipe (via id_funcionario). Exatamente um dos dois deve estar
-- preenchido para cada usuário (regra de negócio a ser reforçada
-- por CHECK ou trigger na Fase 2, junto com o RLS).
CREATE TABLE usuario_acesso (
    id_usuario INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email VARCHAR(150) UNIQUE NOT NULL,
    senha_hash VARCHAR(255) NOT NULL, -- hash gerado no backend (bcrypt/argon2) -
nunca senha em texto puro
    id_nivel_acesso INT NOT NULL REFERENCES nivel_acesso(id_nivel_acesso),
    id_participante INT UNIQUE REFERENCES participante(id_participante), -- preenchido
só para nível 'voluntario'
    id_funcionario INT UNIQUE REFERENCES funcionarios(id_funcionario), -- preenchido
só para 'admin'/'pesquisador'
    ativo BOOLEAN NOT NULL DEFAULT TRUE,
    criado_em TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    ultimo_login TIMESTAMPTZ
);

```

```

CREATE INDEX idx_usuario_acesso_email ON usuario_acesso(email);

```

```

--
=====
-- SEÇÃO 7: DADOS DE REFERÊNCIA (SEED DE PADRONIZAÇÃO)
--
=====
-- Popula as tabelas de domínio da Seção 1 com os valores fixos que o
-- sistema usa. IDs de status_participante, programa e
-- cargo_participante são citados por número em outras partes deste
-- arquivo (comentários e trigger) - não altere esses IDs sem também
-- atualizar as referências.
--
=====

```

```

INSERT INTO status_participante (id_status, nome_status, situacao) VALUES
    (1, 'Ativa', TRUE),
    (2, 'Inativa', FALSE),
    (3, 'Afastada', FALSE), -- pausa temporária, controle manual da equipe (NÃO tocado
pela trigger da Seção 8)
    (4, 'Desligada', FALSE); -- sem nenhum vínculo de programa ativo (mantido
AUTOMATICAMENTE pela trigger)

```

```

INSERT INTO estado_civil (descricao) VALUES
    ('Solteira'),
    ('Casada'),
    ('Divorciada'),
    ('Viúva'),

```

('União estável');

```
INSERT INTO programa (id_programa, nome_programa) VALUES
(1, 'Remama'),
(2, 'OncoFit'),
(3, 'Ambos');
```

```
INSERT INTO cargo_participante (id_cargo, nome_cargo) VALUES
(1, 'Aluna'),
(2, 'Gerente'),
(3, 'Capitã'),
(4, 'Dragonete');
```

```
INSERT INTO status_presenca (sigla, descricao) VALUES
('P', 'Presente'),
('F', 'Falta'),
('J', 'Justificada');
```

```
INSERT INTO funcoes (nome_funcao) VALUES
('Educador Físico'),
('Fisioterapeuta'),
('Administrador'),
('Coordenador'),
('Voluntário de Apoio');
```

```
INSERT INTO nivel_acesso (nome_nivel) VALUES
('admin'),
('pesquisador'),
('voluntario');
```

-- SUGESTÃO - confirmar lista definitiva com a administração do Remama

```
INSERT INTO tipo_comorbidade (nome_comorbidade) VALUES
('Hipertensão'),
('Diabetes tipo 2'),
('Linfedema'),
('Osteoporose'),
('Hipotireoidismo'),
('Depressão/Ansiedade');
```

-- SUGESTÃO - confirmar lista definitiva com a administração do Remama

```
INSERT INTO tipo_treinamento (nome_treinamento) VALUES
('Remada em barco-dragão'),
('Treinamento funcional'),
('Alongamento e mobilidade'),
('Treino de força (OncoFit)'),
('Condicionamento aeróbico');
```

-- SUGESTÃO - confirmar lista definitiva com a administração/equipe clínica do Remama

```
INSERT INTO tipo_cancer (nome_cancer) VALUES
```

```
('Carcinoma ductal invasivo'),  
( 'Carcinoma lobular invasivo'),  
( 'Carcinoma ductal in situ'),  
( 'Câncer de mama triplo negativo'),  
( 'Outro');
```

```
--
```

```
=====
```

```
-- SEÇÃO 8: TRIGGER DE INTEGRIDADE
```

```
-- status_participante (global) <-> inscricao_programa.data_desligamento (por programa)
```

```
--
```

```
=====
```

```
--
```

```
-- O PROBLEMA QUE ESTA TRIGGER RESOLVE
```

```
-- O banco tem DUAS informações parecidas, mas com granularidades  
-- diferentes:
```

```
--
```

```
-- 1. participante.id_status
```

```
-- -> Status GLOBAL: essa participante tem QUALQUER vínculo  
-- ativo com o Remama, em qualquer programa?
```

```
--
```

```
-- 2. inscricao_programa.data_desligamento
```

```
-- -> Status POR PROGRAMA: quando (se) essa participante saiu  
-- de UM programa específico.
```

```
--
```

```
-- Uma participante pode estar inscrita em mais de um programa (ex:
```

```
-- Remama E OncoFit, duas linhas em inscricao_programa). Se ela sai
```

```
-- só de um deles, o status GLOBAL dela (participante.id_status)
```

```
-- deve continuar "Ativa", porque ela ainda participa do outro
```

```
-- programa. Só deve virar "Desligada" quando NÃO sobrar nenhuma
```

```
-- inscrição ativa.
```

```
--
```

```
-- Sem esta trigger, seria responsabilidade de cada ponto do código
```

```
-- da aplicação lembrar de verificar e atualizar os dois lugares
```

```
-- toda vez que uma inscrição for encerrada - um esquecimento gera
```

```
-- dado inconsistente (ex: participante marcada "Ativa" mas sem
```

```
-- nenhuma inscrição de fato ativa). Colocar a regra na trigger
```

```
-- garante a integridade DENTRO DO BANCO, independente de qual
```

```
-- aplicação/script está inserindo os dados.
```

```
--
```

```
-- O QUE A TRIGGER FAZ, PASSO A PASSO
```

```
-- 1. Identifica qual participante foi afetada pela alteração em
```

```
-- inscricao_programa (funciona tanto para INSERT quanto UPDATE
```

```
-- quanto DELETE, por isso o COALESCE entre NEW e OLD).
```

```
-- 2. Verifica se essa participante AINDA tem pelo menos uma
```

```
-- inscrição com data_desligamento IS NULL (ou seja, ativa).
```

```

-- 3. Compara com o status atual gravado em participante.id_status:
--   - Se ela tem inscrição ativa mas estava marcada como
--     "Desligada" (id_status = 4) -> volta para "Ativa" (1).
--   - Se ela NÃO tem mais nenhuma inscrição ativa mas ainda
--     estava marcada como "Ativa" (id_status = 1) -> muda para
--     "Desligada" (4).
-- 4. NÃO mexe em nenhum outro status (ex: "Afastada" = 3). Esse
--   valor é controle MANUAL da equipe (pausa temporária,
--   independente de inscrição em programa) e a trigger
--   propositalmente não sobrescreve essa decisão humana.
--
-- QUANDO A TRIGGER DISPARA
-- AFTER INSERT -> nova inscrição criada (participante pode voltar
--   a ficar "Ativa" se estava "Desligada")
-- AFTER UPDATE OF data_desligamento -> desligamento ou reativação
--   de uma inscrição existente
-- AFTER DELETE -> remoção de uma inscrição (raro, mas coberto)
--
-- Repare que o gatilho de UPDATE é limitado à coluna
-- data_desligamento (UPDATE OF data_desligamento) - a trigger não
-- dispara à toa em updates de outras colunas de inscricao_programa
-- (ex: mudança de turma), evitando processamento desnecessário.
--
=====

CREATE OR REPLACE FUNCTION fn_atualiza_status_participante()
RETURNS TRIGGER AS $$
DECLARE
    v_id_participante INT;
    v_tem_inscricao_ativa BOOLEAN;
    v_status_atual INT;
BEGIN
    -- COALESCE cobre os 3 tipos de evento: em INSERT/UPDATE existe
    -- NEW; em DELETE só existe OLD.
    v_id_participante := COALESCE(NEW.id_participante, OLD.id_participante);

    -- Existe hoje, para esta participante, pelo menos uma inscrição
    -- sem data de desligamento (ou seja, ainda ativa em algum
    -- programa)?
    SELECT EXISTS (
        SELECT 1 FROM inscricao_programa
        WHERE id_participante = v_id_participante
        AND data_desligamento IS NULL
    ) INTO v_tem_inscricao_ativa;

    SELECT id_status INTO v_status_atual
    FROM participante WHERE id_participante = v_id_participante;

```

```

IF v_tem_inscricao_ativa AND v_status_atual = 4 THEN
    -- Tinha sido marcada "Desligada" (sem nenhum vínculo ativo),
    -- e agora surgiu uma inscrição ativa nova -> volta a "Ativa".
    UPDATE participante SET id_status = 1 WHERE id_participante = v_id_participante;

ELSIF NOT v_tem_inscricao_ativa AND v_status_atual = 1 THEN
    -- Estava "Ativa", e a última inscrição ativa que restava
    -- acabou de ser encerrada -> não sobra nenhum vínculo, então
    -- passa para "Desligada".
    UPDATE participante SET id_status = 4 WHERE id_participante = v_id_participante;
END IF;

-- Trigger do tipo AFTER ... FOR EACH ROW não precisa (nem pode,
-- de forma útil) retornar uma linha modificada - o retorno é
-- ignorado pelo Postgres nesse tipo de trigger.
RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_atualiza_status_participante
AFTER INSERT OR UPDATE OF data_desligamento OR DELETE ON inscricao_programa
FOR EACH ROW
EXECUTE FUNCTION fn_atualiza_status_participante();

-- Como testar esta trigger isoladamente (não faz parte do schema,
-- é só um roteiro de verificação manual):
--
-- INSERT INTO participante (cpf, data_nascimento, id_status)
-- VALUES ('11111111111', '1990-01-01', 1);
-- INSERT INTO inscricao_programa (id_participante, id_programa)
-- VALUES (1, 1);
--
-- UPDATE inscricao_programa SET data_desligamento = CURRENT_DATE
-- WHERE id_participante = 1;
--
-- -- deveria retornar id_status = 4 (Desligada), automaticamente:
-- SELECT id_status FROM participante WHERE id_participante = 1;

```